

# 《东方秽漫洋》简要介绍文档

——同时也是某比赛提交作品的说明文档

队长（主催）：RhN03-lx

队员：粘鼎, baiABC

Version:v1.00

## 一、项目亮点提要

这是一个将 STG 与大鱼吃小鱼玩法融合的 2d 游戏的《东方 project》系列的同人游戏

从游戏进行方式来看：

1. 玩家可以吸收啃咬判定范围小于自己的敌人，
2. 玩家需要同时躲避敌人发射的弹幕，自己也可以发射弹幕或使用技能来应敌
3. 关卡流程、特定敌人会呈现出特定行为与机制，需要玩家付出一定的思考与操作来解决

问题

从剧情设定来看：

玩家扮演的是一个从外界流入幻想乡<sup>1</sup>的智能体，因为她自身具备的某些性质而能够与幻想乡内的某些东西互相作用，从哲学意义上拥有了生命与灵智。我们的作品讲述的便是她产生意识之后，在幻想乡内，与这里的人与妖怪发生意想不到的相互作用，对自己的生存与存在产生进一步理解的故事。

有关游戏开发方式：

采用经典的引擎封装+游戏系统+关卡对象行为流程的分层设计思路（这也正是主流游戏引擎做出的游戏的设计范式）。引擎使用 LuaSTG Sub，通过编写 lua 脚本来构建游戏系统，关卡内容的个性化定义使用 LuaSTG Editor Sharp 编辑器。

有关我们作品的创新点，以及富有想象力的部分：

1. 深度融合并联系两种游戏机制。这也许可以从主线内容的演示视频中看出，例：

（1）玩家需要充分同时利用吃与射击这两种机制才可以取得较大收益——快要被大体型敌人堵死的时候需要通过射击来解围，但只依赖射击又会爆出很多死尸弹。

（2）敌人的射击引入了玩家的博弈决策——如果不主动提前吃掉会发子弹的敌人，避弹压力会提高；上前去吃又要当心不要被突然射出的弹幕糊住。

现有的游戏基本进行方式存在较大的拓展空间，也便于我们后续综合运用弹幕设计与谜

---

<sup>1</sup> 这是《东方 project》系列作品故事发生的地点，从设定上看，可以理解为一个与现实世界存在联系的异世界

题设计的手段进一步丰富游戏内容。

2. 深度融合剧情设定与游戏机制。这主要从主线内容的剧情细节可以看出。随着关卡的进行，主角的身体与立绘会逐渐实化，会渐渐摆脱一开始的无口状态。前几个 stage 那些光球，碎片形状敌人寓意着什么？为什么前两个 stage 死亡的时候不允许续关？这些在剧本中预留好的悬念与解释会在完整作品中展现。

3. 与上述两点相适配的创意美术设计。

有关具体表现，请见下文。

## 二、剧情设定、游玩方法、游戏系统简介

### （一）、基本操作方式：

Shift：（按住）切换到低速

Z：射击（如果当前关卡流程允许）、确定（在菜单界面）

X：释放技能（在后续关卡中会启用）、取消（在菜单界面）

C：在符卡练习模式中切换难度

↑ ↓ ← → 方向键：移动

### （二）、基本游玩方法：

见项目内容概览，以及游戏内流程提示

### （三）、游戏系统：

· 标题界面：

可以通过：

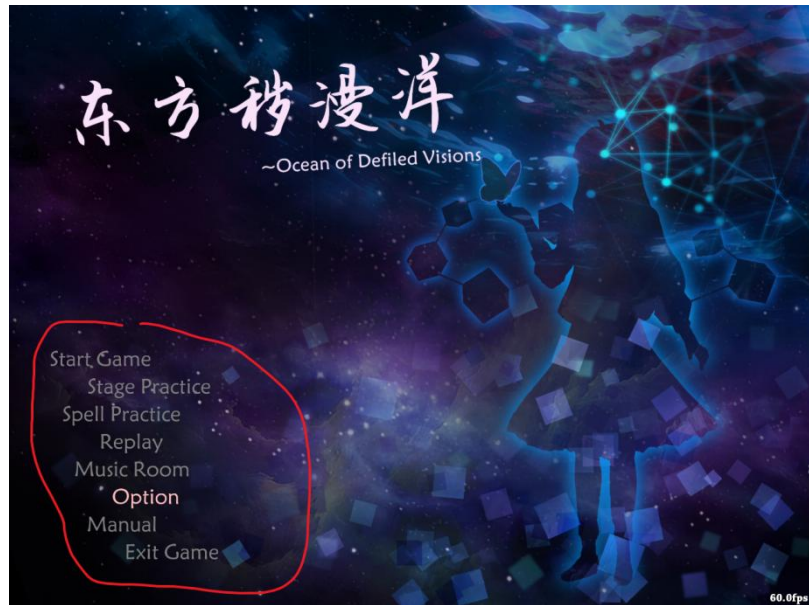
replay 查看回放

option 更改设置，

spellcard practice、stage practice 进入练习模式

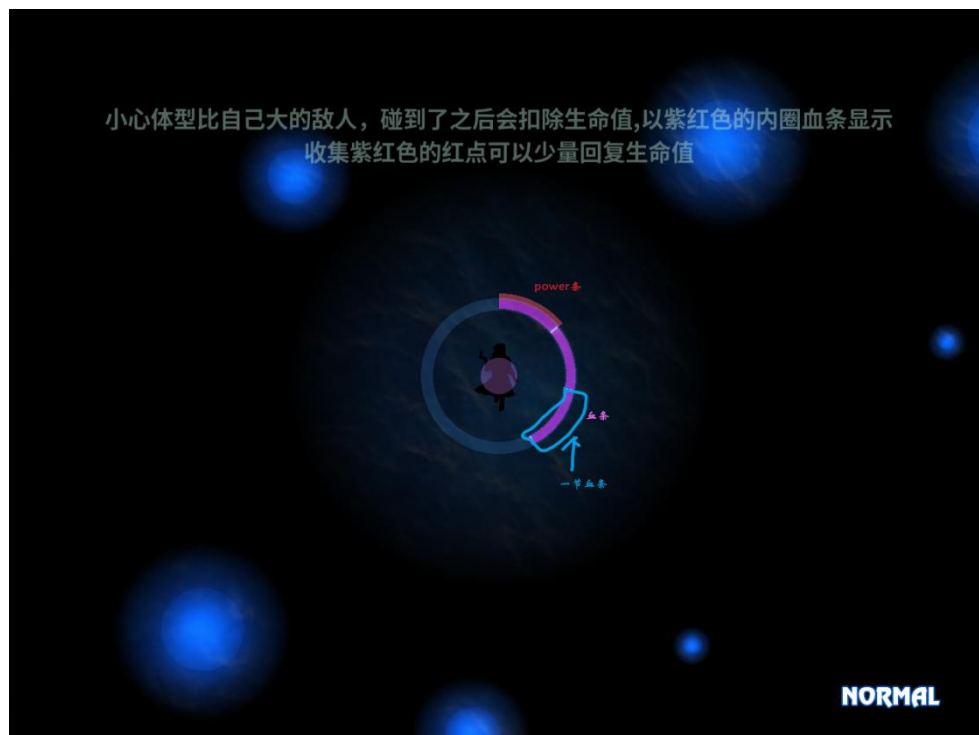
manual 查看游戏介绍

Music room 查看乐评，欣赏游戏 ost



#### · 游戏内

(1) 资源：每收集 100 个紫点，生命值增长一节；每收集 100 个 power，外层的 power 指示条回满一圈，火力等级提升一级（多出来一个子机）

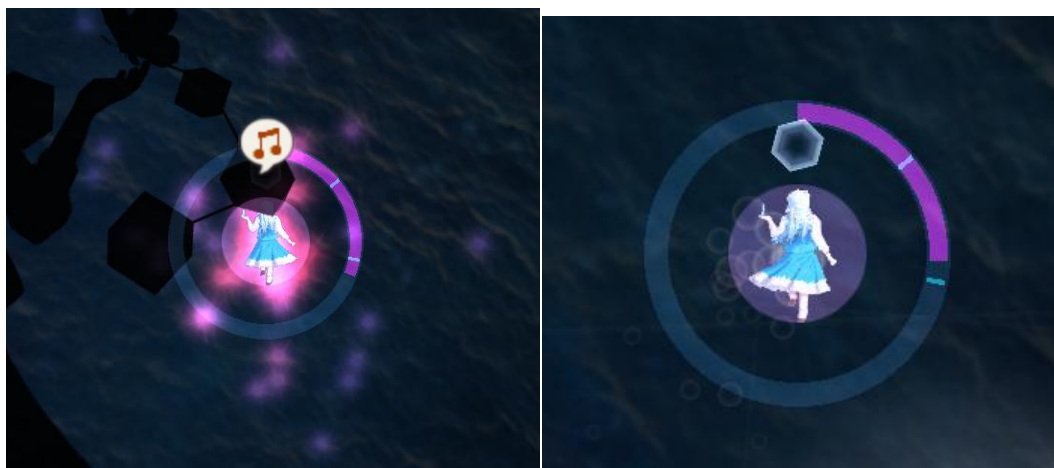


(2) 特殊状态：通过粒子效果来表示

(a) 弱化：持续减少 power

(b) 恢复：持续增加血量

(c) 强化：在一定时间内自身弹幕伤害提高



特殊效果：恢复，弱化

#### 剧情设定与玩法的联系（内含剧透，请谨慎观看）：

（四）、有关故事发生的背景，详情见附录：剧情背景。简要概括之：主角是：外界某个被遗忘的智能体项目、与之相伴的被遗忘的好奇与疑问的集合体，因为幻想乡内存在适合孕育幻想与生命的条件，这里的事物被允许定义自己的存在与意义，被允许有感知与探索真实世界的权利。所以流入幻想乡的她，诞生生命与思维的火种。

#### （五）、有关游戏中所描绘的主角的故事：

• 角色立绘与行走图：因为一开始只是若干信息与数据科学工具碎片的几何体，仅有些微的意识与行动，于是主角一开始是无口、只能用表情气泡、任务行走图颜色极暗的一坨黑影。随着吸收与当前环境有关的知识，建立起对其的感知，角色才渐渐变得“更像个人”。也因为渐渐了解到与幻想乡有关的知识，于是会在 **stage2** 的开头直觉驱动地将现有的 **power** 充分利用，来使自己具备发射弹幕的能力。

#### • 敌人行为与外形：

（1）**各色的虚化光球**寓意各种属性不同的数据。有能增加自己决策视野的（蓝球，扩大视野范围），有质量较高能弥补缺陷的（绿球，回复生命值），有使训练结果倒退的噪声（灰球，持续掉 **power**）。**凝聚成完整实体的碎片**寓意稍微具备一点意识与行动能力的其他智能体（会发射弹幕的碎片），**有引力或斥力的敌人**寓意训练算法给出的粗糙估计所指示的优化方向。完全顺着力的方向行动很有可能败北。

（2）**被体型较大的敌人吞噬**寓意变成了另外一个数据集 or 智能体的一部分，并且在实际的模型训练过程中，一次性投入大量数据容易让模型的决策陷入混乱，不如随机小批次样本训练。

（3）**打爆敌人**寓意以暴力手段强行搅乱 or 摧毁某个系统，会将其裂解为更小的结构单

元（少量 p 点或紫点），有概率崩出有害的片段。

前两个 stage 不允许续关是因为：这里的数据和智能体不像幻想乡的居民那样遵守符卡规则，死了就是死了，没有弥补的机会。只能等待下一个从芜杂中涌现出生命奇迹的机会。

（4）和幻想乡内居民的弹幕决斗，引入了 boss 战系统

详细剧情以及题解，见“剧情设定.pdf”

### 三、技术细节简介

\*只是希望游玩游戏本体的同志请忽略这部分，直接打开 bin/game/LuaSTGSub.exe 运行即可

由于 luastg 内容过于丰富，本部分仅做简要介绍

#### 1. 项目结构

· 引擎层：采用 LuaSTG Sub 引擎提供的封装好的基本图形学操作接口、对象回收与创建接口、对象回调函数、日志系统等基本 API，只需使用 Lua 语言定义对象行为、关卡流程等游戏内容相关的部分，这会由引擎内置的 Lua 虚拟机运行，在此过程中调用 Lua API。

这部分源代码见 LuaSTG Sub 的开源仓库：

<https://github.com/Legacy-LuaSTG-Engine/LuaSTG-Sub>

· Lua 脚本系统层：这些 Lua API 的使用方法教程见 LuaSTG AfterEx 中的示例项目（其中还附带一些实用工具），仓库地址：

<https://github.com/Legacy-LuaSTG-Engine/Bundle-After-Ex-Plus。>

考虑到这个示例项目并不足以完全满足我们的开发需求，遂在此基础上做了大量的修改，并顺带纠正了其中大量语义含混、作用不明、逻辑混乱的代码。

（1）资源文件：公用美术、音乐等资源存储在 bin\game\packages\thlib-resources 中。自机，背景子模块美术资源见 plugin 中

（2）引擎的主要运行逻辑主要对应项目文件中 bin\game\packages\thlib-scripts。

(i)通过 core.lua 加载所有模块并启动游戏，随后通过执行 THlib.lua 导入所有模块

(ii)游戏主体场景：借助 lib/Lstage.lua 系统，游戏被分为两个主要的界面，游戏标题界面（在 THlib\title.lua 中被定义），游戏主体（由编辑器导出，存放在 mod 文件夹中）。这两个界面的各种逻辑接口主要包含在 THlib 文件夹中。

(iii)游戏基本视口(lib/Lscreen.lua),object 系统(lib/Lobject.lua),task

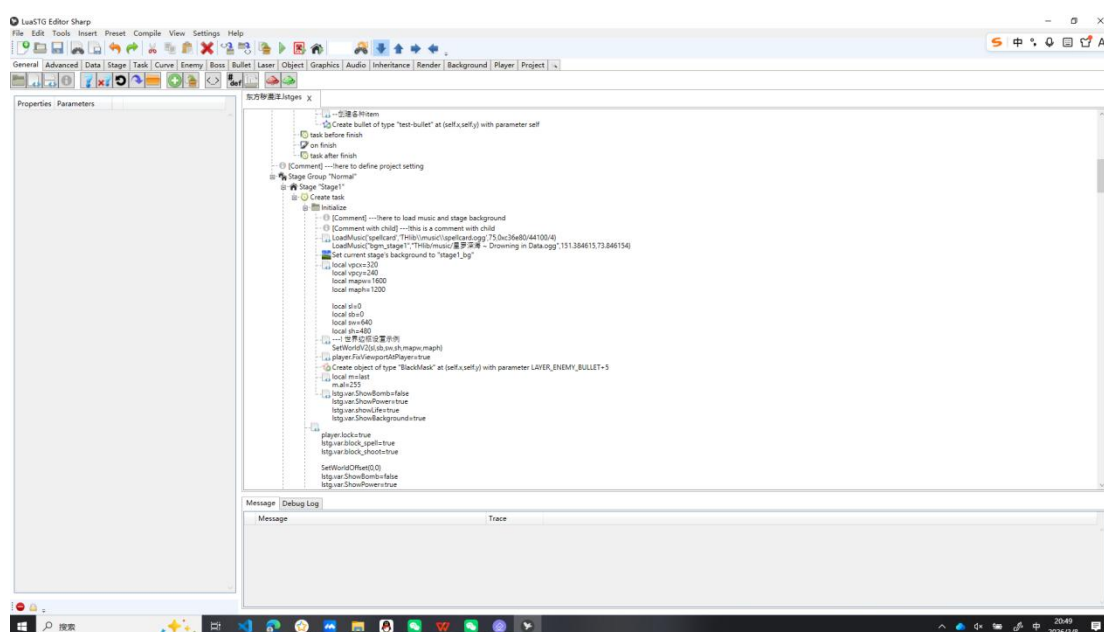
包装器（**Ltask.Lua**）等与各类事件运行有关的基本模块见 **lib** 文件夹。与文件操作和基本场景操作有关的基本模块见 **foundation** 文件夹

(iv)启动游戏后，创建标题界面对应的 **ui** 对象，调用的 **UI** 系统包含在 **UI** 文件夹中。我们也在 **此** 定义各类菜单项的行为。

(v)游戏系统：在 **thlib-scripts/THlib** 文件夹下的若干脚本定义各类预留好的游戏对象，以及可扩展游戏对象的基类（**enemy**，**boss**，**background** 等）。其主要实现包括角色系统（**player** 文件夹），游戏机制系统（**item**，**enemy**，**bullet**，**player** 文件夹等，**ext** 中的碰撞组注册，**replay** 系统，暂停系统，**stagegroup** 系统）以及常用图形渲染预设部分（**background** 文件夹的 **3D** 效果及 **customized-extension** 中的功能扩展）。两者之间的切换逻辑主要在 **ext** 文件夹中定义。

(vi)背景、自机行为的具体定义由 **plugins** 中的 **bg\_attri** 和 **player\_attri** 文件夹中的模块脚本来定义。游戏整体状态通过 **lstg.var** 这个 **table** 中的全局变量以供查询。

· **编辑器层**：关卡流程的设计采用 **LuaSTG Editor Sharp** 编辑器辅助，用以便捷地按模板创建对象、定义关卡流程，设计弹幕上有较为直观的辅助作用。其与游戏引擎的对接主要通过其内置的翻译器及 **THlib\editor.lua** 实现。



编辑器界面截图

项目中 **mod** 文件夹下存放了我们游戏流程中的对象定义与关卡流程脚本，这部分脚本即由编辑器辅助生成。项目结构见下



## 2. 编译、运行方法

### · 引擎编译

参考“三→项目结构”中 LuaSTG Sub 开源仓库的 `build.md`, 配置好编译环境（注意 Visual Studio 版本只能是 2022）后, 用 `cmake` 在不同平台下使用相应 `preset` 编译生成 `LuaSTGSub.exe`(推荐: `cmake --workflow --preset windows-amd64-release` 完成编译), 若失败可删除 `build` 文件夹后重试。

由于我们并未对引擎层做任何修改, 所以也可直接下载标准版的 LuaSTG Sub 引擎（如果你不需要对引擎底层的 API 做修改的话）。可直接在这里下载发行版的引擎（本项目使用 LuaSTG Sub v0.21.109）。

<https://github.com/Legacy-LuaSTG-Engine/LuaSTG-Sub/releases>。

获取到 LuaSTG Sub 引擎后, 直接将其复制粘贴到 `src/script` 下, 可直接运行的项目结构见下图。

| 名称                  | 修改日期            | 类型       | 大小       |
|---------------------|-----------------|----------|----------|
| mod                 | 2026/3/8 15:19  | 文件夹      |          |
| packages            | 2026/2/17 10:14 | 文件夹      |          |
| plugins             | 2026/3/6 23:51  | 文件夹      |          |
| userdata            | 2026/2/11 11:07 | 文件夹      |          |
| config.json         | 2026/2/14 10:12 | JSON 源文件 | 3 KB     |
| LuaSTGSub.exe       | 2026/2/12 18:29 | 应用程序     | 5,855 KB |
| d3dcompiler_47.dll  | 2025/4/20 16:36 | 应用程序扩展   | 4,802 KB |
| xaudio2_9redist.dll | 2026/2/12 18:29 | 应用程序扩展   | 948 KB   |

#### · Lua 脚本运行

将项目结构调整到上图之后，可以直接运行 **LuaSTGSub.exe** 打开整体游戏项目

#### · 弹幕脚本工程文件

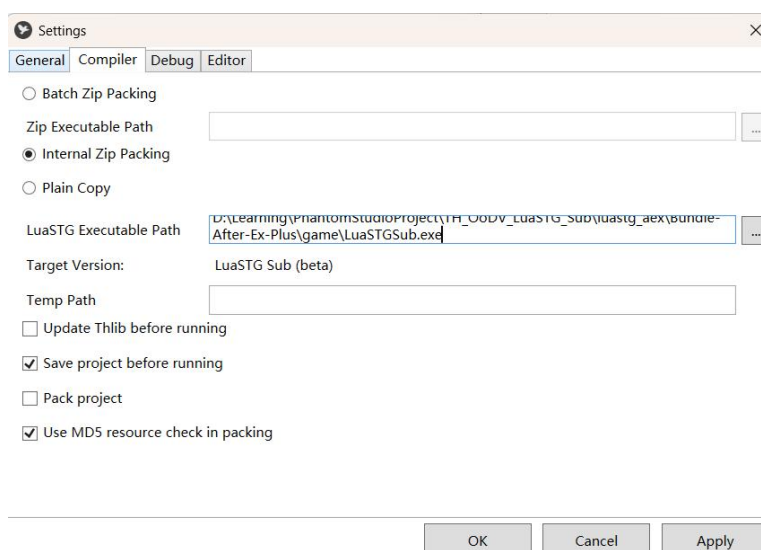
上图中的 **mod** 文件夹内存有的即为 **lua** 弹幕脚本，理论上来说可以直接阅读。但在实际工程实践中，出于便利，我们采用 **LuaSTG Editor Sharp** 编辑器自带的图形化编辑器来定义常见对象行为。工程文件见 **src/editor project/东方秽漫洋 ~ Ocean of Defiled Visions.lstges**。

为了查看该工程文件，操作步骤见下：

(1). 预先配置 **LuaSTG Editor Sharp** 编辑器. 你可以通过访问 **sharp editor** 的 **github** 仓库来获取最新版的编辑器：

<https://github.com/czh098tom/LuaSTG-Editor-Sharp/releases>

获取到编辑器后，打开 **LuaSTGEditorSharp.exe**，菜单栏选择 **Settings-Compiler Settings**，在 **LuaSTG Executable Path** 一栏找到 **LuaSTBSub.exe**，然后点 **Apply**。



(2).本项目加载与运行方法:打开 LuaSTGEditoraSharp.exe,菜单栏 File-Open 打开“东方秽漫洋.lstges”,再点击菜单栏 Compile-pack project,最后点击菜单栏 Compile-Run (或按快捷键 F5)即可运行游戏程序。

